

Clothes Segmentation

CVCY002

A 9-class semantic segmentation model (U-Net++), trained from scratch

0.553 overall mIoU	0.502 foreground mIoU	0.675 mean Dice	90.7% pixel accuracy	38 epochs trained
------------------------------	---------------------------------	---------------------------	--------------------------------	-----------------------------

1 Dataset and Preprocessing

Dataset. The model is trained on the **People Clothing Segmentation** dataset (Kaggle, CC0): 1,000 images paired with pixel-level masks across 59 original classes, at a uniform resolution of 825×550. It was preferred over the alternatives considered — iMaterialist Fashion 2020 (RLE-encoded masks, roughly 24 GB), Clothing Co-Parsing (.mat-format masks), ModaNet (polygon annotations with no bundled images), and DeepFashion (bounding boxes only, no pixel masks) — because it is the only one that ships ready-to-use PNG image/mask pairs requiring no format conversion.

Per the assignment brief, no pretrained encoders were used: both architectures implemented (build_unet and build_unet_plus_plus in model.py) are trained entirely from randomly-initialized weights.

Pipeline. data_preprocessing.py matches images to masks by filename id and splits the data 80 / 10 / 10 into train, validation, and test sets. Both images and masks are resized to a fixed 256×256 size — images using bilinear interpolation, masks using nearest-neighbor interpolation exclusively, since interpolating class-id integers would invent invalid intermediate class values. data_augmentation.py then applies segmentation-safe augmentation to the training split only (validation and test splits are left unaltered, since they must reflect real, unmodified inputs): horizontal flip and random crop-and-resize are applied identically to image and mask, while brightness, contrast, and saturation jitter are applied to the image alone. All mask-affecting resize operations use nearest-neighbor interpolation exclusively.

Class remapping (59 → 9 classes). An initial model trained on all 59 original classes reached an overall mIoU of only 0.143, with more than 20 classes stuck at exactly 0.0 IoU (ring, watch, wallet, tie, hoodie, heels, loafers, gloves, bodysuit, and others). With roughly 1,600 training images after augmentation spread across 59 classes, most fine-grained garment types did not have enough examples to learn distinguishable boundaries. class_mapping.py therefore groups the 59 original classes into 9 semantic superclasses, applied to the full-resolution mask before resizing so the remap lookup only ever sees valid original class ids:

Superclass	Original classes folded in
background	null
upper_clothes	blazer, blouse, bra, cape, cardigan, coat, hoodie, intimate, jacket, jumper, shirt, sweater, sweatshirt, t-shirt, top, vest
lower_clothes	jeans, leggings, panties, pants, shorts, skirt, stockings, tights
dress	bodysuit, dress, romper, swimwear
shoes	boots, clogs, flats, heels, loafers, pumps, sandals, shoes, sneakers, socks, wedges
accessories	accessories, bag, belt, bracelet, earrings, glasses, gloves, hat, necklace, purse, ring, scarf, sunglasses, tie, wallet, watch
skin	skin
hair	hair

suit	suit
------	------

Suits were kept as their own class rather than folded into dress, upper_clothes, or lower_clothes, since a suit mask typically spans torso and legs as a single annotated region and does not cleanly correspond to any of the other groups. This remapping trades fine-grained garment-type recognition (e.g. distinguishing a jacket from a coat) for substantially more reliable coarse-category segmentation — an appropriate tradeoff given the dataset size, though a real limitation discussed further in Section 6.

2 Model Architecture

Two architectures were implemented from scratch in `model.py`: a standard U-Net baseline, and U-Net++ (Zhou et al., 2018), which is the model used for the final reported results. U-Net++ replaces each single skip connection with a grid of nested, densely-connected convolution blocks; at encoder depth level i and nesting position j :

$$X_{i,j} = \text{Conv}(\text{Concat}(X_{i,0}, X_{i,1}, \dots, X_{i,j-1}, \text{Upsample}(X_{i+1,j-1})))$$

This means the decoder at each resolution receives a dense concatenation of all previously-computed feature maps at that level, rather than a single matching encoder skip. The intent — following the original paper — is to progressively close the semantic gap between shallow, fine-detail features and deep, coarse-semantic features before fusion, helping recover finer object boundaries. Clothing segmentation involves exactly this kind of fine-boundary challenge: sleeve edges, collar lines, and the boundaries between adjacent garments.

Deep supervision was intentionally omitted — only the final, most-nested output node produces the prediction, with no auxiliary losses at intermediate decoder stages — to keep the implementation simpler and easier to debug and explain. This is a deliberate simplification rather than an oversight (see Section 6). The final architecture used feature widths [32, 64, 128, 256] (four levels), chosen over the paper’s five-level default to control parameter count given the roughly 1,600-image training set.

3 Loss Function

The loss is a weighted hybrid of Dice loss and class-weighted cross-entropy:

$$\text{total loss} = 0.7 \times \text{Dice loss} + 0.3 \times \text{weighted CE loss}$$

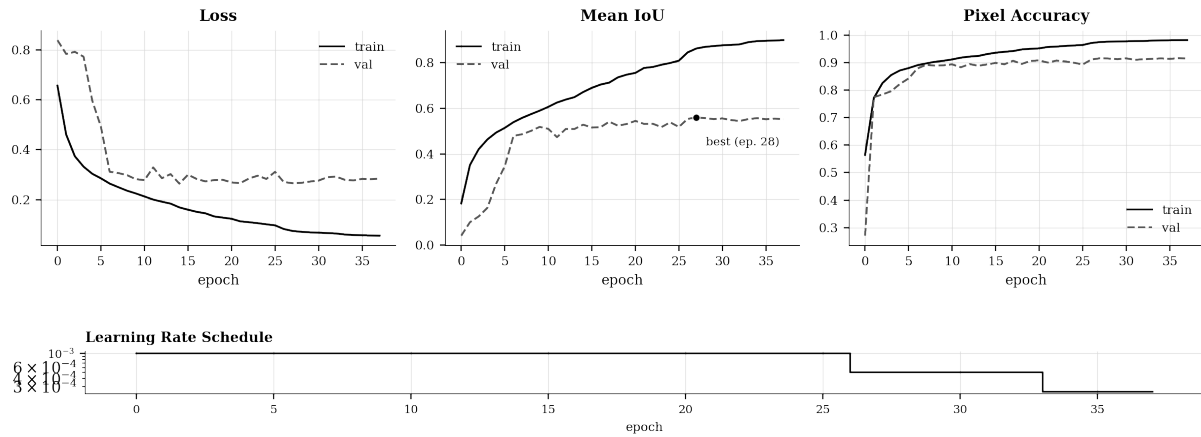
Dice loss directly optimizes for region overlap, closely related to IoU, which is more informative than pixel-wise accuracy for a task with severe class imbalance — a single image’s clothing pixels can be a small fraction of the total compared to background and skin. Cross-entropy, weighted by inverse log-frequency per class, provides more stable early-training gradients than Dice alone, which can behave poorly when predictions are near-random.

A bug found and fixed during development. The initial Dice loss implementation averaged per-class Dice scores over all classes, including classes with zero ground-truth pixels in a given batch. Because softmax never outputs an exact zero probability, an absent class’s predicted-probability sum was always slightly positive while its intersection was exactly zero, driving that class’s Dice score to roughly 0.32 rather than the correct 1.0, even for a mathematically perfect prediction. This was confirmed directly: a synthetic perfect-prediction test case produced a Dice loss of 0.655 under the buggy implementation, where it should have been approximately zero. With five to ten classes typically present per image out of the full class count, this meant the majority of every batch’s per-class loss terms were near-meaningless noise, diluting the gradient signal on the classes that actually mattered.

The fix restricts averaging to only the classes with at least one ground-truth pixel present in the batch. The same synthetic test case now correctly returns a loss of approximately zero. The fix was verified to remain fully differentiable before being adopted for all subsequent training runs, including the final reported results.

4 Training Results

Training ran for 38 of a 60-epoch budget; early stopping and checkpoint selection both monitor `val_mean_iou`, with `ModelCheckpoint(save_best_only=True)` retaining the epoch-28 weights rather than the final epoch’s.

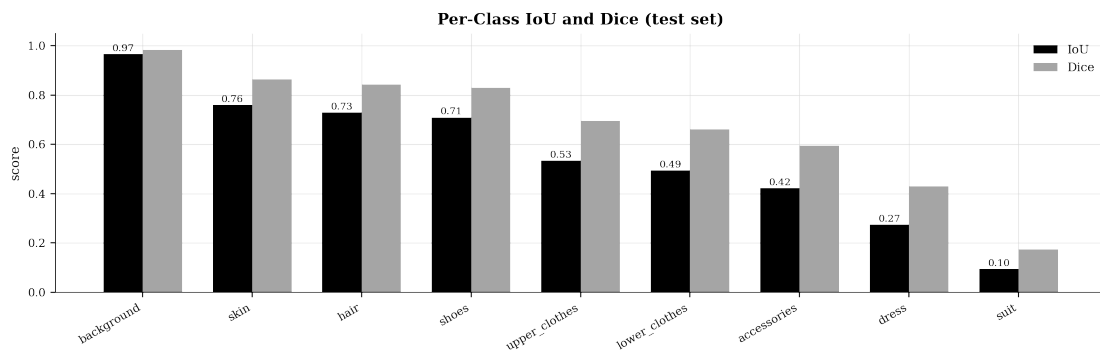


Metric	Value
Best val_mean_iou (epoch 28)	0.560
val_accuracy at best checkpoint	0.910
Final-epoch train_mean_iou	0.899
Final-epoch train_loss / val_loss	0.055 / 0.284

The gap between final-epoch train_mean_iou (0.899) and val_mean_iou (0.552–0.560) indicates the model was overfitting by the later epochs — expected given the small dataset and the absence of a pretrained encoder. Retaining the epoch-28 checkpoint rather than the final epoch’s weights protects the reported results against this.

5 Test-Set Evaluation

`eval.py` reports five metrics on the held-out, 100-pair test split: overall mIoU (mean IoU across all classes present); foreground mIoU (the same, excluding background, so a large and trivially easy region does not inflate the headline number); mean Dice (macro-averaged per-class Dice, computed the same way as the training loss for a fair comparison); pixel accuracy; and the full per-class IoU/Dice breakdown, shown below.



6 Sample Predictions

The figure below shows six of the 100 test-set examples: input image, ground truth, and model prediction.



Predictions capture the overall silhouette and pose well, but show visible noise inside garment regions — speckled misclassification between adjacent clothing classes — and softer, less precise boundaries than ground truth, consistent with the fine-boundary weaknesses discussed in Section 6.

7 Strengths, Weaknesses, and Limitations

Strengths. The model separates large, high-contrast regions reliably: skin, hair, and broad garment categories (upper and lower clothes) show strong IoU, consistent with the goal of the class remapping. It is reasonably robust to moderate variation in pose and background clutter, aided by geometric and color augmentation on the training set, and it has no dependency on pretrained weights or external model downloads at inference time.

Weaknesses. Because of the 59-to-9 class remapping, the model cannot distinguish between garment types within a superclass — for example, a t-shirt and a sweater are both `upper_clothes`. This was a deliberate tradeoff for reliability given the dataset size, but it limits applicability wherever garment-type-specific output is required. The train/validation mean_iou gap (0.899 vs. approximately 0.55) by the later epochs reflects the constraint of training from scratch on roughly 1,600 images with no pretrained encoder, a constraint specified by the assignment rather than a preprocessing choice. The omission of deep supervision trades some of U-Net++’s potential accuracy for implementation simplicity, and the fixed 256×256 input resolution loses fine details such as thin straps, small accessories, and garment texture, a practical choice made to fit training within available GPU memory.

Operating limitations. The dataset and model assume a single person per image; multi-person scenes are untested and likely to produce unreliable results. Training data is predominantly frontal-facing fashion photography, so extreme poses, partial occlusion, or unusual camera angles fall outside the training distribution. No explicit testing was done under low light, strong shadows, or backlit conditions, and the model was trained and evaluated only on real photographs — performance on drawings, renders, or heavily stylized images is unknown.

Reproduce these numbers with: `uv run scripts/eval.py --run-name <your-run-name>`, which produces `models/<run-name>/eval/metrics.json` and `sample_predictions.png`.